BUILDING STATISTICAL MODELS

PIPELINE DEEP LEARNING

8

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Estimativa de criação de dados na Internet em um minuto 5
Figura 2 – Tipos de dados e suas proporções 6
Figura 3 – Relação entre Inteligência Artificial, Machine Learning e Deep Learning.7
Figura 4 8
Figura 5 – Diferença entre Machine Learning e Deep Learning 8
Figura 6 – O impacto de DL em visão computacional, com resultados da ImageNet 10
Figura 7 – Comparação entre GPU e CPU para realizar o mesmo cálculo matricial, de acordo com o tamanho da matriz 12
Figura 8 – Algumas aplicações “clássicas” de Deep Learning 13
Figura 9 – Algumas aplicações “modernas” de Deep Learning.....14
Figura 10 – O que as CNNs estão aprendendo? 15
Figura 11 – Componentes típicos de uma CNN 16
Figura 12 – Comparação de uma RNN (esquerda) com uma rede feedforward (direita) 17
Figura 13 – Comparação de uma RNN com LSTM e GRU 18
Figura 14 – Representação de uma GAN 19
Figura 15 – Arquiteturas de Redes Neurais (Profundas ou não).....22
Figura 16 – Ciclo de vida de soluções baseadas em Ciência de Dados 23
Figura 17 – Exemplo de representação vetorial de texto 25
Figura 18 – Exemplo de representação vetorial de imagem 26
Figura 19 – Exemplo de aumentação de dados para imagens 27
Figura 20 – Exemplo de aumentação de dados para texto 28
Figura 21 – Exemplo de aumentação de dados para texto 29
Figura 22 – Saída da execução do código-fonte 1 32
Figura 23 – Saída da execução do código-fonte 4 33
Figura 24 – Operação de achatamento de uma imagem 34
Figura 25 – Saída da execução do código-fonte 9 36
Figura 26 – Saída da execução do código-fonte 10 37
Figura 27 – Saída da execução do código-fonte 12 – Parte A 43
Figura 28 – Saída da execução do código-fonte 12 – Parte B 43

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Imports necessários.....31
Código-fonte 2 – Carregando o dataset Fashion MNIST 32
Código-fonte 3 – Explicitando o nome dos labels do dataset.....32
Código-fonte 4 – Exibindo algumas imagens do dataset 33
Código-fonte 5 – Normalizando os dados 34
Código-fonte 6 – Normalizando os dados 35
Código-fonte 7 – Rede neural construída com Sklearn.....35
Código-fonte 8 – Rede neural construída com TensorFlow 35
Código-fonte 9 – Resumindo a rede neural construída com TensorFlow.....36
Código-fonte 10 – Treinando a rede neural construída com TensorFlow.....37
Código-fonte 11 – Treinando a rede neural construída com TensorFlow.....37
Código-fonte 12 – Possível solução do desafio Hands On 42

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

PIPELINE DE DEEP LEARNING 5
1 INTRODUÇÃO 5
1.1 Qual a necessidade de novos algoritmos de aprendizado de máquina? 5
1.2 Conceitos e Histórico de Deep Learning 7
1.3 Viabilizadores e aplicações 11
1.3.1 O poder computacional das GPUs 11
1.3.2 Novos algoritmos 13
2 PRINCIPAIS ARQUITETURAS 15
2.1 Redes Neurais Convolucionais 15
2.2 Redes Neurais Recorrentes 16
2.3 Redes Adversárias Generativas 19
2.4 Outras arquiteturas 20
3 ETAPAS DE DESENVOLVIMENTO 23
3.1 Comparação com Redes Neurais Multicamadas 23
3.2 Preparação de dados 24
3.3 Aumentação de dados 27
4 FRAMEWORKS DE DEEP LEARNING 28
5 MÃO NA MASSA! DEEP LEARNING EM PYTHON 30
6 HANDS ON: PRATICANDO O USO DE TENSORFLOW 39
6.1 Aquecimento 39
6.2 Seu desafio 39
6.3 Proposta de solução 40
REFERÊNCIAS 44
GLOSSÁRIO 45

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

PIPELINE DE DEEP LEARNING

1 INTRODUÇÃO

1.1 Qual a necessidade de novos algoritmos de aprendizado de máquina?

No mundo moderno, somos banhados por um oceano de dados. A cada segundo, uma quantidade exorbitante de informações é gerada, capturada e compartilhada em todas as esferas da nossa sociedade digitalizada. Desde transações comerciais até interações em redes sociais, passando por registros médicos e dispositivos de IoT, os dados fluem incessantemente, inundando nossos sistemas de armazenamento e desafiando nossa capacidade de compreensão.

Para ilustrar a produção de dados na era da Internet, a Figura 1 mostra a quantidade estimada de dados em um minuto, de acordo com o aplicativo/plataforma, para o ano de 2021. Imagine como esses números aumentaram até hoje!

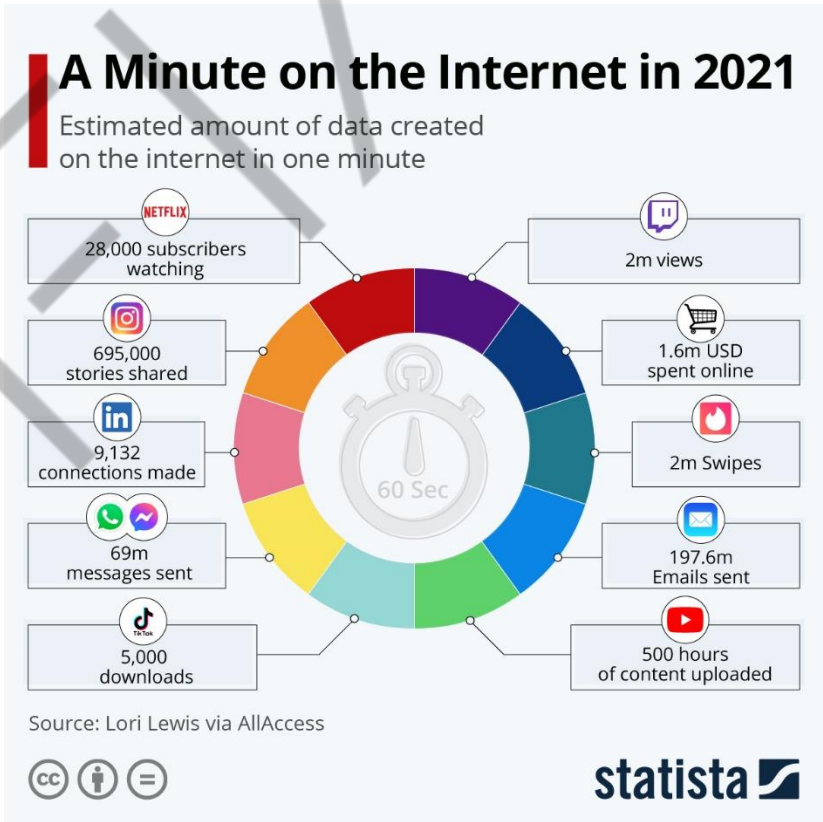

Figura 1 – Estimativa de criação de dados na Internet em um minuto
Fonte: Imagem reproduzida de https://www.statista.com/chart/25443/estimated-amount-of-data-created-on-the-internet-in-one-minute/

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Entretanto, a maior parte dos dados gerados e compartilhados é complexa e não estruturada. Estima-se que cerca de 80% de todos os dados produzidos sejam não estruturados, representando imagens, vídeos, áudios, textos, e-mails, postagens em redes sociais, entre tantas outras formas e fontes. A proporção e fontes de dados estão ilustrados na Figura 2.

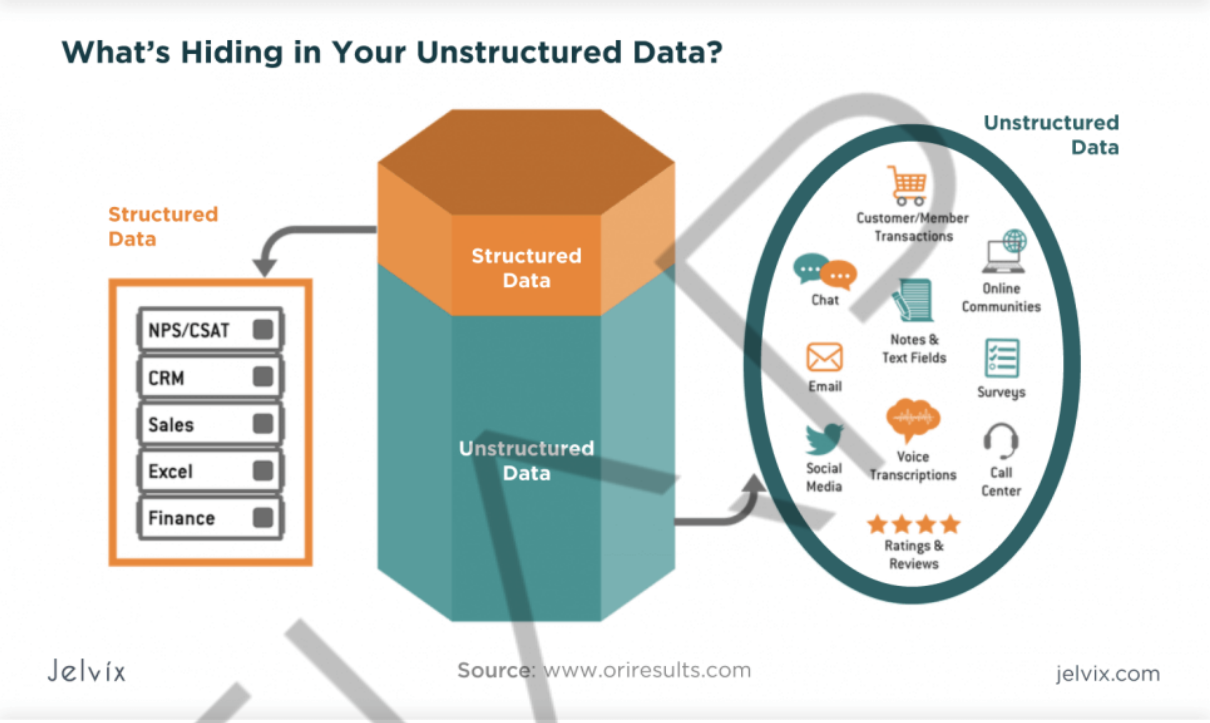

Figura 2 – Tipos de dados e suas proporções
Fonte: Imagem reproduzida de https://jelvix.com/blog/structured-vs-unstructured-data

Toda essa complexidade e robustez resiste às técnicas tradicionais de processamento e análise de dados, exigindo abordagens mais sofisticadas para extrair informações significativas. Nesse contexto, surge uma pergunta crucial: **como lidar com essa avalanche de dados e transformá-la em conhecimento acionável?**

É nesse ponto que entra em cena a necessidade premente de novos algoritmos de aprendizado de máquina. Enquanto os métodos tradicionais podem ser eficazes para lidar com dados estruturados e tarefas relativamente simples, eles muitas vezes sucumbem diante da complexidade e da escala dos dados não estruturados. Para desvendar os insights ocultos nessas montanhas de dados, precisamos de ferramentas mais poderosas, capazes de aprender padrões sutis e extrair informações valiosas de maneira eficiente e escalável. É aqui que o **Deep Learning** (DL) entra em

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

cena, oferecendo promessas de descobertas revolucionárias em um mundo inundado de dados.

1.2 Conceitos e Histórico de Deep Learning

Deep Learning (DL) (Aprendizado Profundo) é uma subárea do campo de **Inteligência Artificial** (IA) que se concentra no treinamento de modelos de **Machine Learning** (ML) (Aprendizado de Máquina) em conjuntos de dados muito grandes e complexos. Aqui, vale relembrar a relação entre os conceitos, reforçado na Figura 3. É devido a essa relação que o treinamento de modelos de DL segue os mesmos passos e estratégias de ML, por exemplo, separação entre treino e teste, etapas de validação, cálculo de métricas sob o conjunto de teste, entre outros.

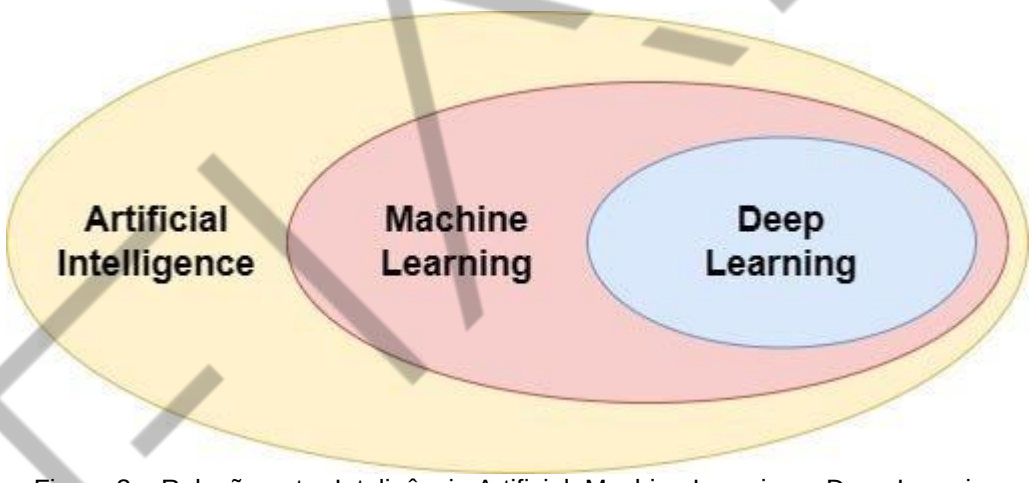

Figura 3 – Relação entre Inteligência Artificial, Machine Learning e Deep Learning
Fonte: Imagem reproduzida de https://medium.com/@matheus1714/diferen%C3%A7a-de-ia-ml-e-dl-6f1a06882859

Os modelos de aprendizado de máquina que servem de base para o Deep Learning são **Redes Neurais Artificiais** (RNAs). A principal diferença entre o DL e as RNAs, é que o primeiro emprega diversas camadas ocultas para aprender representações e correlações dos dados (Figura 4).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

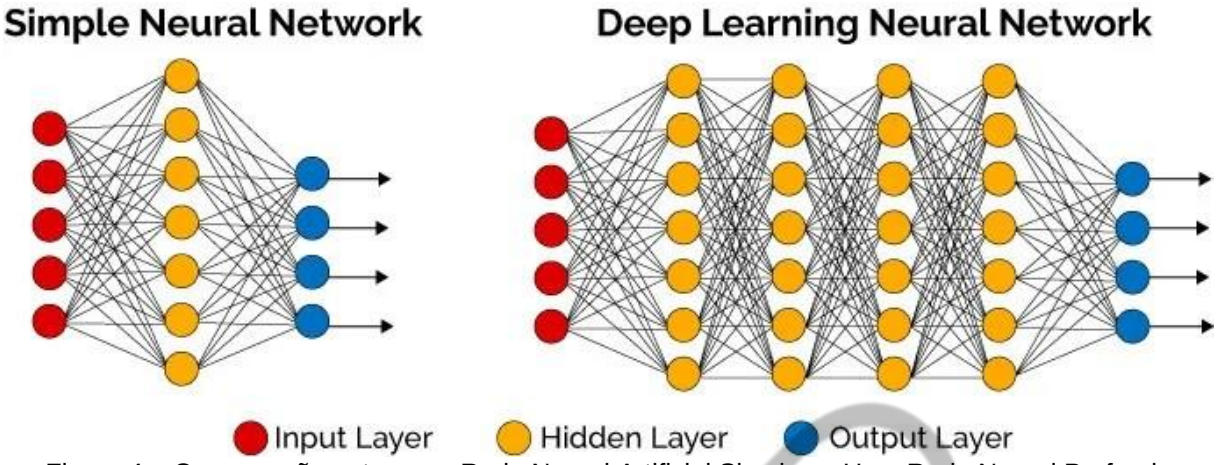

Figura 4 – Comparação entre uma Rede Neural Artificial Simples e Uma Rede Neural Profunda
Fonte: Imagem reproduzida de https://thedatascientist.com/what-deep-learning-is-and-isnt/

Ao contrário de métodos tradicionais de aprendizado de máquina, que dependem de características cuidadosamente projetadas e extraídas manualmente dos dados, o **Deep Learning aprende automaticamente representações hierárquicas a partir de dados brutos** (Figura 5).

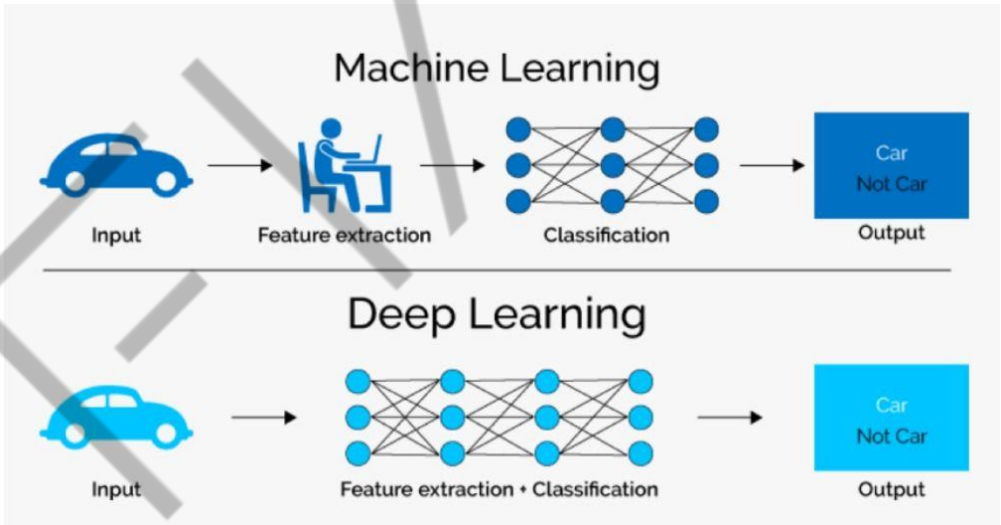

Figura 5 – Diferença entre Machine Learning e Deep Learning
Fonte: Imagem reproduzida de https://kenzie.com.br/blog/o-que-e-machine-learning-e-deep-learning/

Essas representações hierárquicas permitem que os modelos aprendam características cada vez mais abstratas e complexas dos dados, possibilitando a resolução de tarefas extremamente complexas, como reconhecimento de imagens, processamento de linguagem natural e previsão em uma variedade de domínios.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Mas **se engana quem pensa que Deep Learning dispensa as etapas de limpeza e preparação de dados**. Embora os modelos de Deep Learning sejam capazes de aprender características complexas diretamente dos dados brutos, é crucial garantir que os dados de entrada estejam limpos, organizados e preparados adequadamente para garantir o desempenho ideal do modelo. A qualidade dos dados tem um impacto significativo na eficácia e na precisão dos modelos de Deep Learning. Portanto, a limpeza e a preparação de dados continuam sendo etapas essenciais no processo de desenvolvimento de modelos de Deep Learning.

Em essência, o Deep Learning viabiliza o processamento e interpretação de dados armazenados sob tipos complexos, com pouca intervenção humana na criação de features. Em contrapartida, os modelos de DL são ávidos por dados e exigem alto poder computacional para treinamento. Ainda assim, seu crescimento e resultados promissores em análises preditivas ou tomada de decisões abrem portas para uma variedade de aplicações inovadoras em diversas indústrias.

E a trajetória do Deep Learning?

A história do Deep Learning é uma evolução da história das RNAs. Até 2011, as RNAs eram mais uma abordagem de ML, que podiam ou não entregar bons resultados em tarefas de regressão ou classificação.

O marco significativo correu em 2012, na competição ImageNet. A competição ImageNet, ou **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)**, é uma competição anual que visa avançar o estado da arte em visão computacional e reconhecimento de padrões. O desafio é classificar objetos presentes em imagens em milhares de categorias, usando um conjunto de dados extremamente grande e diversificado chamado ImageNet. Tal conjunto possui mais de 14 milhões de imagens divididas em mais de 20 mil categorias.

Nos anos anteriores da competição (2010 e 2011), as performances dos modelos eram modestas (com taxas de ERRO em torno de 28%), mas em 2012, a AlexNet, uma rede neural profunda, obteve um desempenho impressionante, superando os métodos tradicionais por uma margem significativa (Figura 6).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

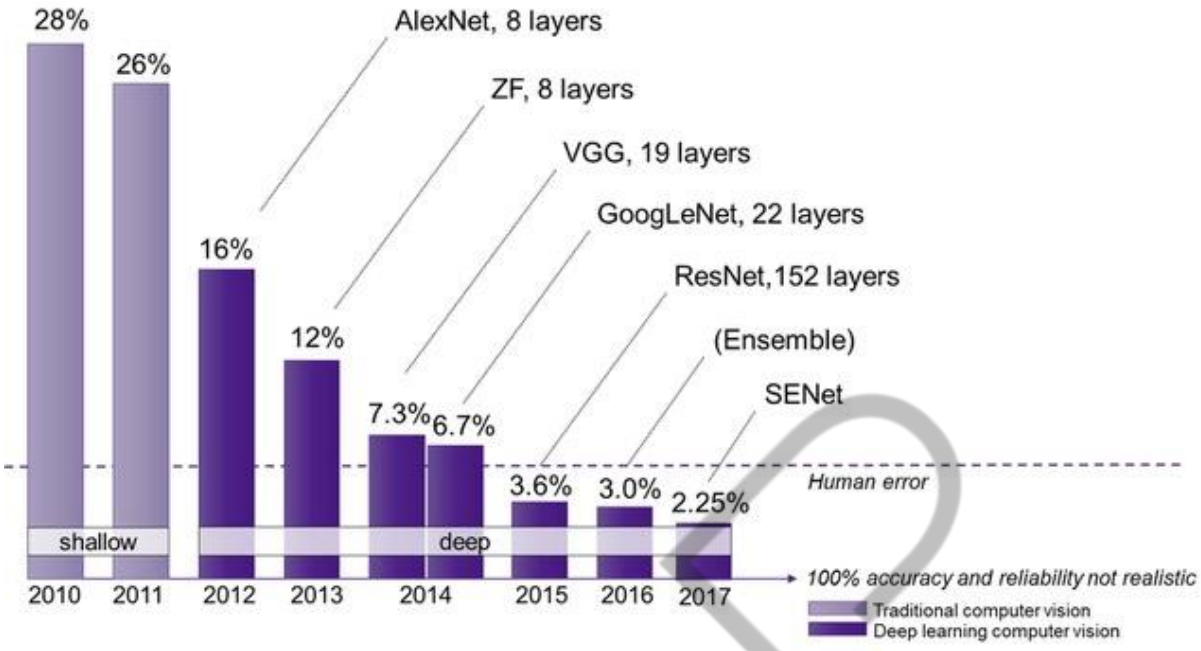

Figura 6 – O impacto de DL em visão computacional, com resultados da ImageNet
Fonte: Imagem reproduzida de https://semiengineering.com/new-vision-technologies-for-real-world-applications/

A AlexNet reduziu o erro de classificação em mais de 10% em relação ao primeiro colocado do ano anterior, demonstrando a capacidade do Deep Learning em lidar com conjuntos de dados complexos e de grande escala.

A Figura 6 mostra, ainda, que a partir de 2012, todos os outros ganhadores da ImageNet propuseram modelos baseados em DL também, cujas taxas de erro superam a capacidade humana de reconhecimento dos objetos das imagens da base. **É curioso notar que uma rede "profunda" em 2012 possuía apenas 8 camadas! Em contraste, o melhor modelo de 2015 possuía 152 camadas!**

A partir do marco introduzido em 2012 pela AlexNet, houve um aumento exponencial na pesquisa e no desenvolvimento de técnicas de Deep Learning, levando a avanços significativos em várias áreas, como visão computacional, processamento de linguagem natural e reconhecimento de voz, nos anos seguintes.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.3 Viabilizadores e aplicações

A ascensão do Deep Learning no início da década de 2010 foi impulsionada por três vários fatores-chave:

- **Avanços computacionais**: permitindo o treinamento eficiente de modelos complexos em grandes conjuntos de dados;
- **Avanços em dados**: acesso a conjuntos de dados massivos e rotulados, como o ImageNet, proporcionou a oportunidade de treinar modelos de Deep Learning em tarefas de reconhecimento de imagens em escala sem precedentes;
- **Avanços de algoritmos**: como Redes Neurais Convolucionais (CNNs) e Redes Neurais Recorrentes (RNNs), que mostraram um desempenho superior em uma variedade de tarefas de inteligência artificial.

1.3.1 O poder computacional das GPUs

Uma GPU (***Graphics Processing Unit*** – Unidade de Processamento Gráfico) é um tipo de processador especializado projetado para lidar com tarefas intensivas em processamento gráfico, como renderização de gráficos 3D, jogos, modelagem de computação visual e transcodificação de vídeo.

As GPUs são altamente eficientes na realização de operações matriciais paralelas, o que é fundamental para o treinamento de redes neurais profundas. As operações matriciais são usadas para calcular as saídas das camadas neurais, aplicar funções de ativação, calcular gradientes durante o processo de backpropagation e ajustar os pesos da rede durante o processo de otimização.

Dado que muitos dos cálculos em Deep Learning envolvem grandes conjuntos de dados e redes neurais com milhões de parâmetros, a capacidade das GPUs de processar operações matriciais em paralelo é essencial para acelerar o treinamento. Isso permite treinar redes neurais profundas em um tempo razoável, tornando a pesquisa e o desenvolvimento de modelos de Deep Learning muito mais eficientes.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O poder computacional das GPUs é frequentemente medido em termos de **FLOPS** (***Floating-point Operations Per Second*** – Operações de ponto flutuante por segundo), que representa a capacidade de realizar cálculos matemáticos por unidade de tempo. Quando comparada com uma **CPU** (***Central Processing Unit*** – Unidade Central de Processamento), a vantagem da GPU é gritante: a Figura 7 mostra a comparação do mesmo cálculo matricial executado numa CPU e uma GPU. Um gigaflop são 10^9 flops.

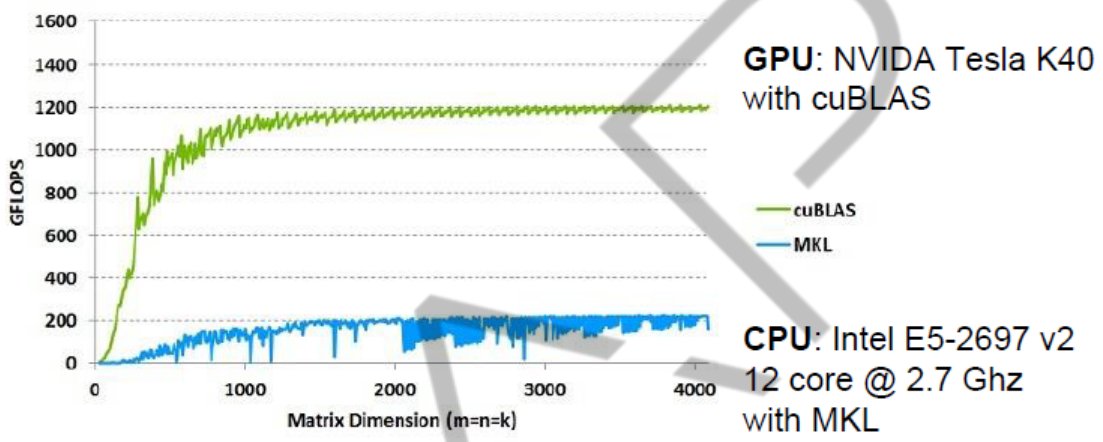

Figura 7 – Comparação entre GPU e CPU para realizar o mesmo cálculo matricial, de acordo com o tamanho da matriz
Fonte: NVIDIA

Observe que à medida que a matriz aumenta (eixo X), a quantidade de operações necessárias para realizar o cálculo também aumenta (eixo Y). Claramente haverá um limite físico de operações que tanto a CPU quanto a GPU podem realizar, mas o limite da GPU é 6 vezes maior do que a CPU (para as placas avaliadas).

Com o aumento significativo do número de FLOPS fornecido pelas GPUs modernas, os cientistas de dados e pesquisadores foram capazes de treinar modelos de Deep Learning em conjuntos de dados massivos em um tempo muito menor do que antes. Isso possibilitou a experimentação rápida e iterativa com diferentes arquiteturas de rede e hiperparâmetros, impulsionando ainda mais a inovação e o progresso no campo do Deep Learning.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.3.2 Novos algoritmos

Para finalizar essa introdução e contextualização, é importante destacar que o termo "Deep Learning" serve como um guarda-chuva para uma variedade de algoritmos de aprendizado de máquina baseados em RNAs profundas.

Cada algoritmo pode levar a uma arquitetura de rede neural diferente, com características únicas, como o número de camadas, neurônios, tipos de neurônios e conexões entre eles.

Essa diversidade de arquiteturas permite que os modelos de deep learning sejam adaptados e otimizados para resolver uma ampla gama de problemas, desde tarefas de processamento de linguagem natural até reconhecimento de imagem, detecção de objetos, previsão de séries temporais e muito mais.

Cada arquitetura pode ser projetada e ajustada de acordo com as características específicas do problema em questão, resultando em modelos altamente especializados e eficazes para diferentes aplicações.

Algumas aplicações “clássicas” de Deep Learning estão na Figura 8, como processamento de vídeos para segurança, reconhecimento de áudio e texto como novas formas de interação com a tecnologia, reconhecimento de objetos, predições de safra, entre outras.

Figura 8 – Algumas aplicações “clássicas” de Deep Learning
Fonte: Autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O avanço de Deep Learning é tremendo, gerando aplicações “modernas” como recomendações de ação para ganho de produtividade, e geração de textos, imagens e vídeos (Figura 9).

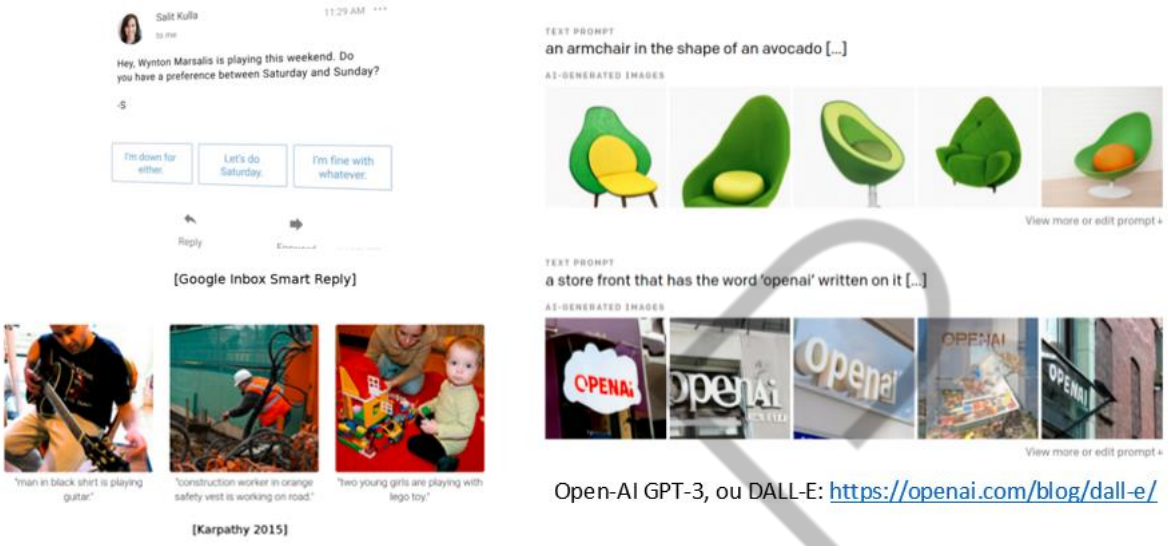

Figura 9 – Algumas aplicações “modernas” de Deep Learning
Fonte: Autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 PRINCIPAIS ARQUITETURAS

Conforme mencionado, existem diversas arquiteturas de Deep Learning. A seguir, introduziremos as principais. Neste momento, não temos a intenção de sermos detalhistas, mas apenas diferenciar os modelos, destacando suas motivações, particularidades e aplicações.

2.1 Redes Neurais Convolucionais

As Redes Neurais Convolucionais (do Inglês "*Convolutional neural network*", CNN), são um tipo especial de rede neural projetada para processar e analisar dados de forma semelhante ao cérebro humano, especialmente quando se trata de imagens e vídeos. Elas são chamadas de "convolucionais" porque utilizam uma operação matemática chamada convolução para processar os dados de entrada.

Essas redes são compostas por várias camadas, cada uma com um conjunto específico de funções. Na camada inicial, a rede recebe a imagem como entrada e aplica filtros para extrair características simples, como bordas e cores. À medida que avança para camadas mais profundas, as características extraídas tornam-se mais complexas e abstratas, permitindo à rede reconhecer padrões cada vez mais sofisticados, como formas, texturas e objetos (Figura 10).

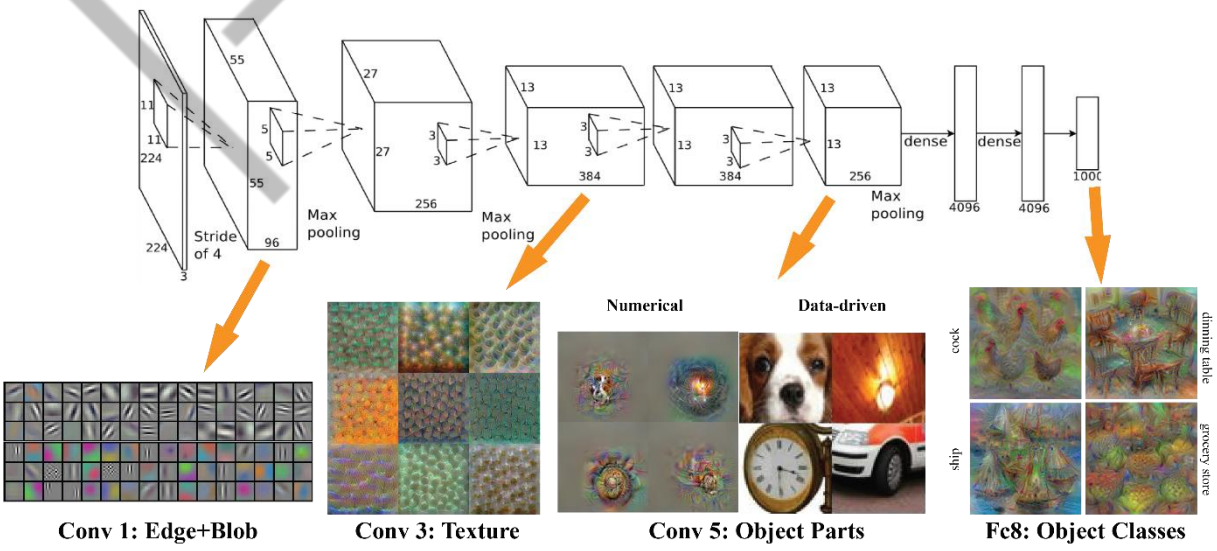

Figura 10 – O que as CNNs estão aprendendo?
Fonte: Imagem reproduzida de: https://donglaiw.github.io/proj/mneuron/index.html

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Uma das características mais importantes das redes neurais convolucionais é a capacidade de compartilhar pesos. Isso significa que os mesmos filtros são aplicados em diferentes partes da imagem, o que reduz significativamente o número de parâmetros a serem aprendidos e torna o modelo mais eficiente. Além disso, as camadas convolucionais são frequentemente seguidas por camadas de pooling, que reduzem a dimensão dos dados, preservando as características mais importantes (Figura 11).

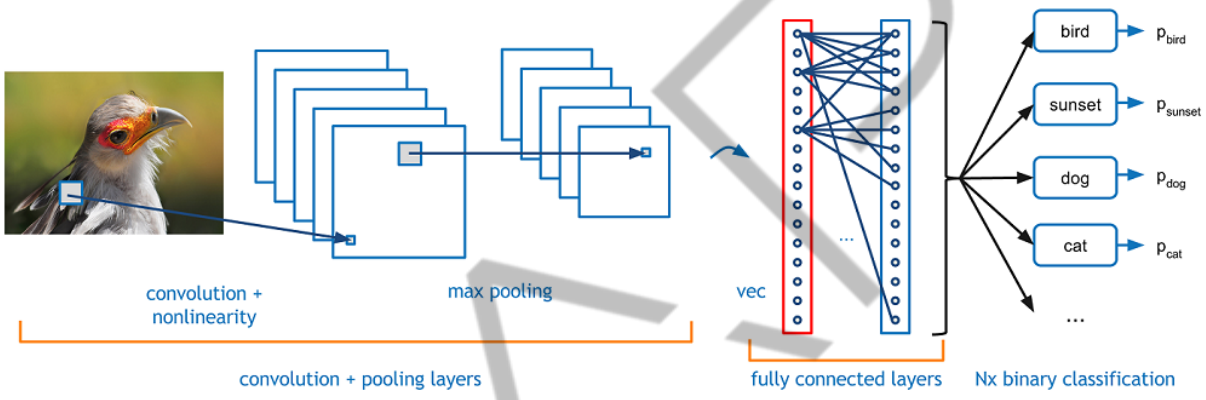

Figura 11 – Componentes típicos de uma CNN
Fonte: Imagem reproduzida de: https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks/

Essa combinação de convolução e pooling permite que as redes neurais convolucionais aprendam representações hierárquicas dos dados, o que é fundamental para seu sucesso em tarefas de reconhecimento de padrões, como classificação de imagens e detecção de objetos.

2.2 Redes Neurais Recorrentes

As Redes Neurais Recorrentes (do Inglês, "*Recurrent Neural Network*", RNNs) são um tipo especial de rede neural projetado para lidar com dados sequenciais, como séries temporais, texto e áudio. Ao contrário das redes neurais convencionais, que processam cada entrada de forma independente, as RNNs possuem loops internos que lhes permitem manter informações sobre o estado anterior enquanto processam

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

a entrada atual. Isso as torna capazes de capturar dependências temporais e contextuais nos dados, tornando-as ideais para tarefas que envolvem sequências de dados.

A principal diferença entre uma RNN e uma rede padrão (feedforward) está ilustrada na Figura 12, que são as arestas representando os loops internos de cada neurônio, os quais simbolizam a memorização da informação:

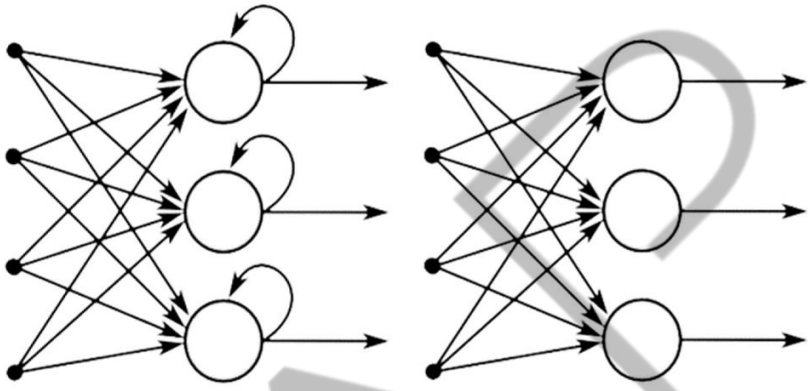

Figura 12 – Comparação de uma RNN (esquerda) com uma rede feedforward (direita)
Fonte: Imagem reproduzida de: https://www.mdpi.com/2079-9292/11/15/2387

A estrutura em loop das RNNs permite que elas processem entradas de comprimento variável e produzam saídas de acordo com o contexto da entrada. Cada passo de tempo em uma RNN recebe uma entrada e uma representação do estado oculto anterior como entrada, e produz uma saída e uma nova representação do estado oculto. Essa capacidade de manter uma memória de longo prazo e atualizá-la dinamicamente ao longo do tempo é fundamental para tarefas como tradução automática, geração de texto, reconhecimento de fala e previsão de séries temporais.

No entanto, as RNNs tradicionais têm algumas limitações, como dificuldade em capturar dependências de longo prazo e propensão ao desaparecimento do gradiente. Para lidar com esses problemas, surgiram variações mais avançadas das RNNs, destacando-se (Figura 13):

- As **Redes Neurais Recorrentes de Memória de Curto Longo Prazo** (do Inglês "*Long Short-Term Memory*", LSTMs) e,
- As **Unidades Recorrentes Fechadas** (do Inglês "*Gated Recurrent Units*", GRUs).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Ambas as arquiteturas são capazes de aprender e reter informações ao longo de sequências muito longas, tornando-as mais eficazes em uma variedade de tarefas de aprendizado de máquina. A principal diferença entre as LSTMs e as GRUs está na sua complexidade e na quantidade de portões de controle que possuem. Sem entrar em muitos detalhes, as LSTMs possuem três portões de controle: o **portão de esquecimento**, o **portão de entrada** e o **portão de saída**, enquanto as GRUs possuem apenas dois portões: o **portão de atualização** e o **portão de reset**. Simplificando, esses portões são responsáveis por estabelecer qual informação continuará retida na rede qual deve ser esquecida.

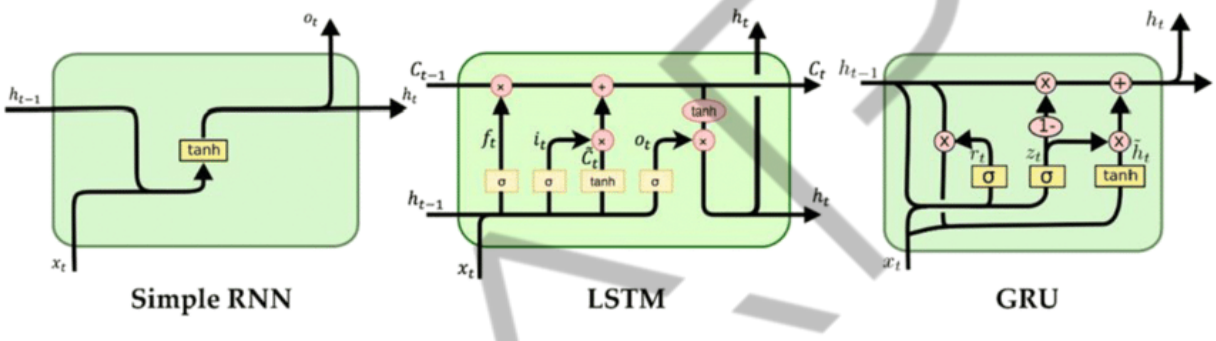

Figura 13 – Comparação de uma RNN com LSTM e GRU
Fonte: Imagem reproduzida de: https://wikidocs.net/166316

As LSTMs são mais complexas e têm mais parâmetros, o que as torna capazes de aprender relações temporais mais longas. No entanto, essa complexidade também pode tornar o treinamento mais lento e propenso a overfitting em conjuntos de dados menores. Por outro lado, as GRUs são mais simples e têm menos parâmetros, o que as torna mais rápidas para treinar e menos propensas a overfitting. No entanto, essa simplicidade também pode limitar sua capacidade de capturar dependências de longo prazo em alguns casos.

As RNNs, juntamente com suas variantes LSTM e GRU, são amplamente utilizadas em aplicações de processamento de linguagem natural (PLN), como análise de sentimentos, tradução automática e geração de texto. Além disso, são empregadas em tarefas de séries temporais, como previsão de séries financeiras e climáticas, e em sistemas de recomendação personalizada, onde são capazes de capturar padrões temporais complexos para fazer previsões mais precisas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.3 Redes Adversárias Generativas

Redes Adversárias Generativas (do Inglês, "*Generative Adversarial Network*", GANs) são um tipo de modelo de Deep Learning composto por duas redes neurais: o **gerador** e o **discriminador**. Essas redes são treinadas em conjunto, com o objetivo de gerar dados que se assemelhem a amostras reais do conjunto de dados original. O gerador cria amostras sintéticas, enquanto o discriminador tenta distinguir entre amostras reais e sintéticas (Figura 14).

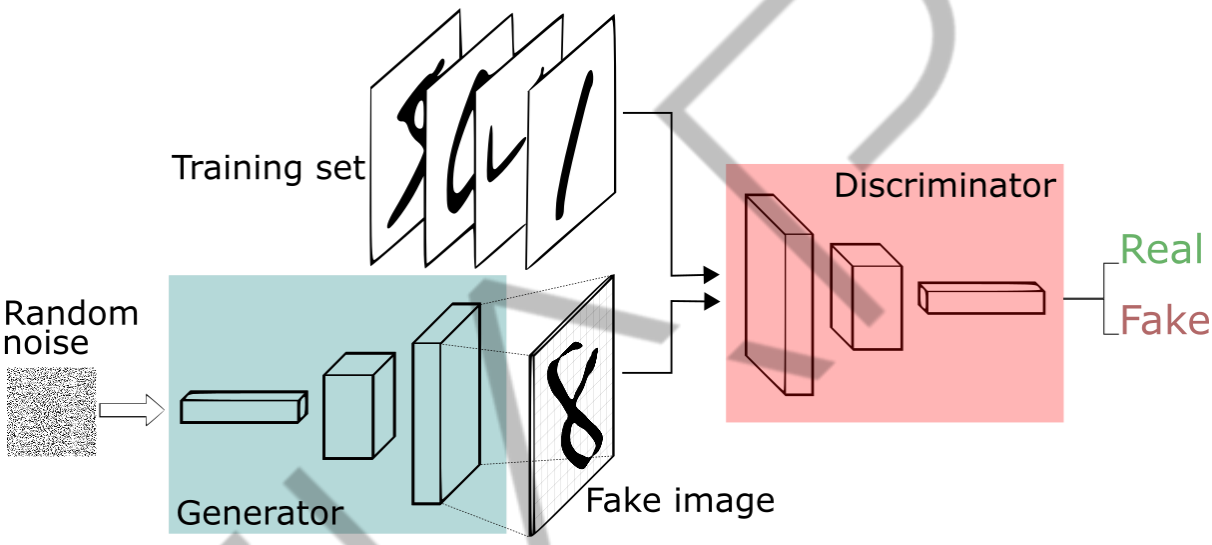

Figura 14 – Representação de uma GAN
Fonte: Imagem reproduzida de: https://www.deeplearningbook.com.br/introducao-as-redes-adversarias-generativas-gans-generative-adversarial-networks/

O processo de treinamento das GANs é iterativo e competitivo. O gerador tenta melhorar constantemente suas habilidades para gerar amostras cada vez mais realistas, enquanto o discriminador busca aprimorar sua capacidade de distinguir entre amostras reais e falsas. Essa competição leva a um equilíbrio dinâmico, onde o gerador aprende a gerar amostras que são cada vez mais indistinguíveis das reais.

As GANs têm uma ampla gama de aplicações, incluindo **geração de imagens**, **síntese de texto**, **geração de áudio** e muito mais. Elas são especialmente úteis em tarefas onde a disponibilidade de dados é limitada ou onde a coleta de dados reais é cara ou impraticável. No entanto, treinar GANs pode ser desafiador e requer técnicas avançadas de otimização e regularização para evitar problemas como colapso do modo e instabilidade durante o treinamento.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Esse tipo de arquitetura de Deep Learning desempenha um papel crucial no avanço da **Inteligência Artificial Generativa** (*Generative AI*, ou *GenAI*), permitindo a criação de conteúdos diversificados e personalizados. Além disso, elas impulsionam o desenvolvimento de novas técnicas de geração de dados e representam um passo significativo em direção à criação de sistemas de Inteligência Artificial capazes de entender e produzir informações complexas e realistas. Por outro lado, a geração de dados levanta questões e amplia o debate sobre seus usos maliciosos (como "*fake news*"), ética em IA e efeitos de vieses de dados.

2.4 Outras arquiteturas

Atualmente, as principais aplicações de Deep Learning se baseiam nas arquiteturas apresentadas. Existem outras arquiteturas profundas além delas, mas sua introdução está fora do escopo. Dentre as demais arquiteturas, destacam-se:

- **Autoencoders**: são redes neurais usadas para aprendizado não supervisionado de representações de dados. Eles consistem em uma rede codificadora que reduz a dimensionalidade dos dados de entrada e uma rede decodificadora que tenta reconstruir os dados de entrada originais a partir da representação codificada;
- **Redes Generativas Adversariais Variacionais (VAE-GAN)**: são uma combinação entre GANs e autoencoders variacionais (VAEs). Elas são capazes de gerar imagens de alta qualidade e oferecem maior controle sobre a geração de amostras;
- **Redes Siamesas**: são arquiteturas que consistem em dois ramos idênticos que compartilham os mesmos pesos. São usadas em tarefas de aprendizado por pares, como reconhecimento facial e tradução;
- **Redes Recorrentes Convolutivas (RCN)**: são uma combinação de redes neurais convolucionais (CNNs) e redes neurais recorrentes (RNNs). Elas são eficazes em modelar sequências temporais em dados 2D, como séries temporais em imagens.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A título de curiosidade, a lista de outras arquiteturas de redes neurais (profundas ou não) e suas representações está na Figura 15. A composição de uma lista completa é praticamente impossível, já que novas arquiteturas são inventadas ou aperfeiçoadas o tempo todo.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

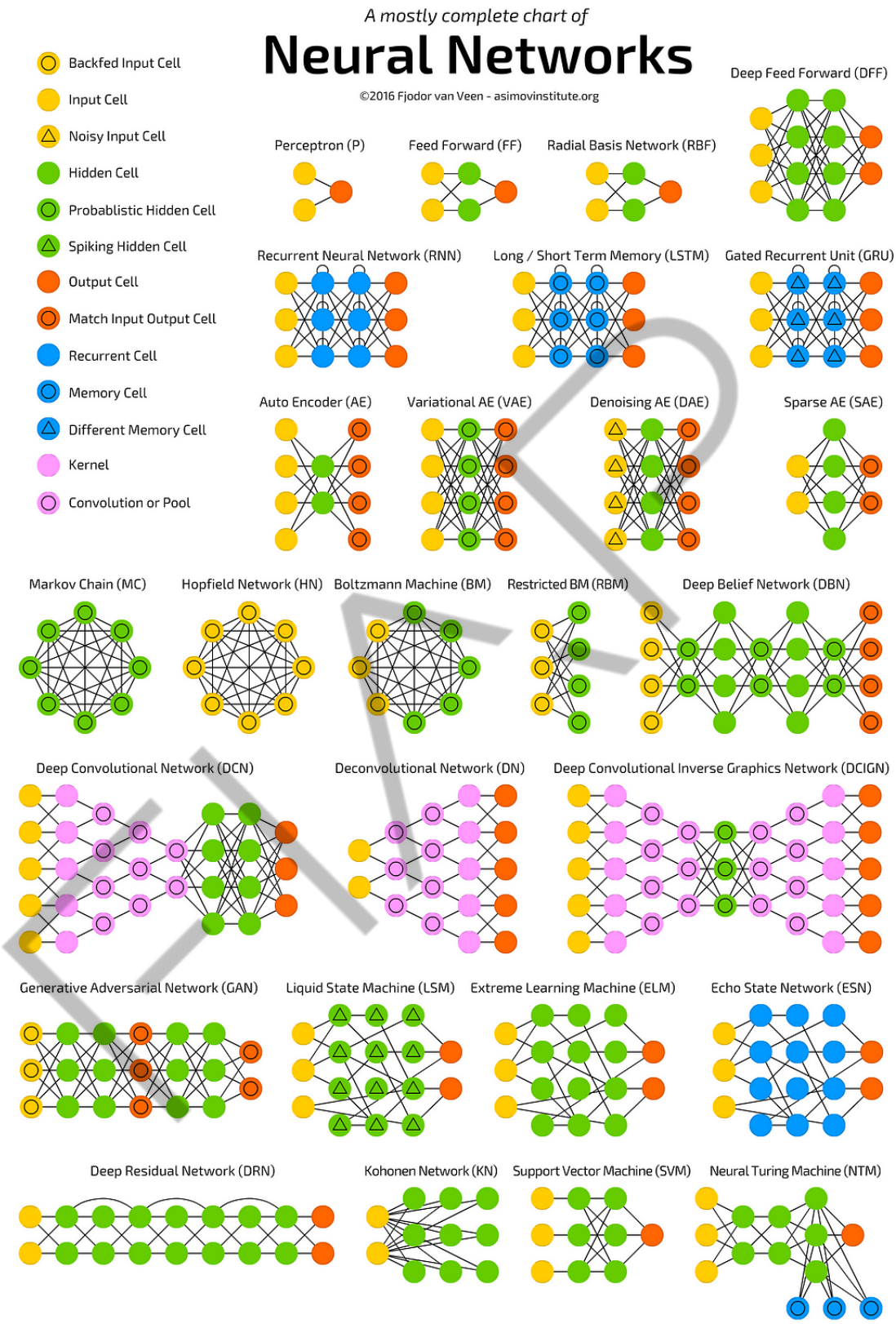

Figura 15 – Arquiteturas de Redes Neurais (Profundas ou não)
Fonte: Imagem reproduzida de: https://www.asimovinstitute.org/author/fjodorvanveen/

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3 ETAPAS DE DESENVOLVIMENTO

3.1 Comparação com Redes Neurais Multicamadas

Ao analisarmos o pipeline de projetos de Data Science, constatamos que os passos a serem seguidos são os mesmos para qualquer projeto preditivo com dados estruturados. Fazer análise de dados não-estruturados, portanto, que requerem modelos profundos, também guarda várias semelhanças com as etapas ilustradas na Figura 16. Por exemplo, ainda é necessário buscar e nos familiarizar com os dados, bem como dividir os dados em treino e teste, definir métricas de avaliação dos modelos, criar modelos iniciais e refinar de forma iterativa.

Figura 16 – Ciclo de vida de soluções baseadas em Ciência de Dados
Fonte: https://medium.com/@vanshikagoel/data-science-life-cycle-in-5-steps-3fe720b2a2fe

A principal semelhança continua sendo no "**entendimento de negócio**", pois é fundamental compreender a importância do modelo preditivo proposto para compreendermos como e por quem ele será utilizado. Essa avaliação guiará todo o processo de desenvolvimento.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

As diferenças aparecem em etapas pontuais, por exemplo:

- **Data Mining**: o processo de coleta de dados é semelhante, buscando-se conjuntos de dados relevantes para o problema em questão. A diferença está na ordem de magnitude dos datasets voltados para Deep Learning, facilmente ultrapassando milhões de exemplos;
- **Data Cleaning** & **Data Exploration**: nas RNAs, a exploração de dados permite entender a distribuição e as relações entre as variáveis de entrada e saída. Além disso, a limpeza é essencial para garantir a qualidade dos dados de entrada e evitar distorções nos resultados. Já em Deep Learning, a limpeza tem um destaque secundário, pois as variações nos dados podem ser importantes para captar nuances da aplicabilidade real, bem como conferir robustez ao método. Por outro lado, a exploração dos dados é importante para conhecer potenciais vieses que podem influenciar incorretamente o treinamento;
- **Feature Engineering**: essa etapa está totalmente voltada para dados estruturados. Já para dados não-estruturados, a criação (manual) de features é dispensada, pois os modelos profundos são construídos para extraírem informações diretamente dos dados.

Em resumo, embora as etapas do pipeline de Data Science sejam semelhantes ao abordar problemas modelados com RNA e DL, algumas diferenças podem surgir devido às características específicas dessas técnicas de modelagem.

3.2 Preparação de dados

Ainda que a extração de features em Deep Learning seja feita de forma transparente pelos modelos, a preparação dos dados ainda é relevante. Essa etapa é crucial para garantir que os dados estejam em um formato adequado para serem utilizados pelos modelos profundos. Isso envolve transformar dados como texto,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

áudio, imagem e vídeo em representações vetoriais, que são o formato padrão para entrada em modelos de Deep Learning.

Por exemplo, no caso de dados de texto, pode-se usar técnicas como tokenização e embedding (Figura 17) para converter palavras em vetores numéricos que podem ser processados pelos modelos.

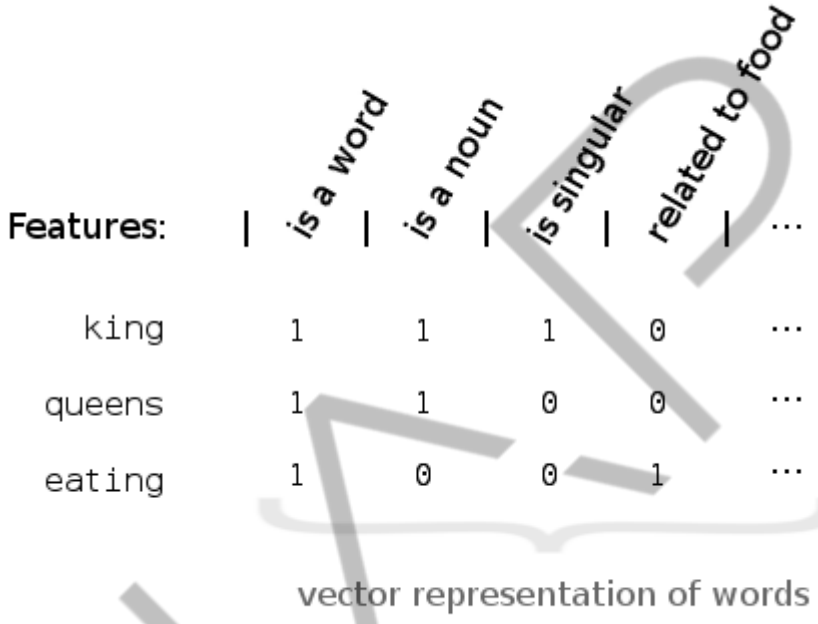

Figura 17 – Exemplo de representação vetorial de texto
Fonte: Imagem reproduzida de https://www.learndatasci.com/tutorials/intro-to-word-embeddings-problems-theory/ https://medium.com/@vanshikagoel/data-science-life-cycle-in-5-steps-3fe720b2a2fe

A Figura 17 mostra como cada palavra (*king*, *queens* e *eating*) pode ser representada por um vetor que sinaliza informações daquela sequência de letras. No caso dessa representação, os verbetes estão sendo caracterizados como palavra, substantivo, singular ou plural, se o significado está associado com comida e por aí vai. Você pode estar imaginando que existem diversas representações (embeddings) para palavras... e isso está correto! Cada representação levará a um vetor de features melhor ou pior para determinada tarefa.

Para dados de imagem e vídeos, técnicas como redimensionamento e normalização de canais podem ser aplicadas para representar os dados de forma vetorial. Pegando por exemplo uma imagem, a matriz de pixels que a compõe pode

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

ser “achatada” para criar uma representação vetorial mais simplificada daquela informação (Figura 18).

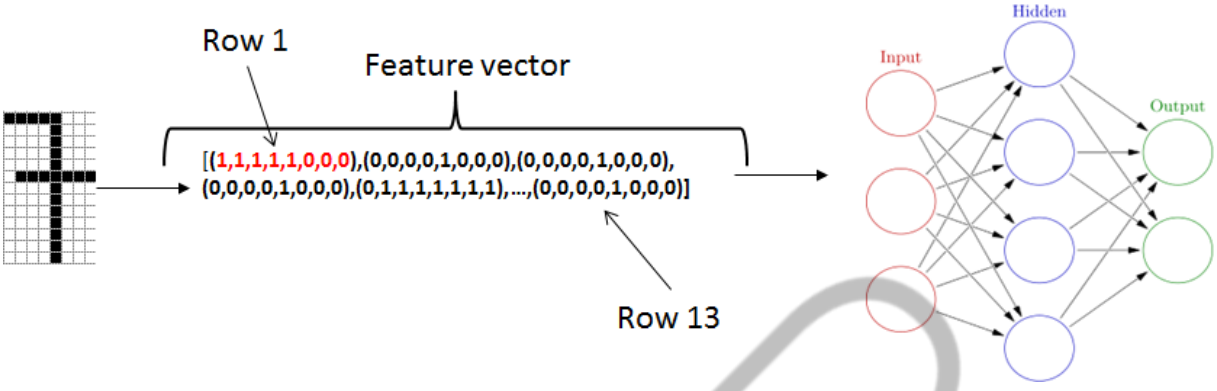

Figura 18 – Exemplo de representação vetorial de imagem
Fonte: Imagem reproduzida de https://stackoverflow.com/questions/27227450/feature-vector-representation-neural-networks

No exemplo da Figura 18, o número “7” está representado como uma matriz de 13 linhas e 8 colunas, completamente preenchida por “0” (ausência de informação), mas com alguns “1” indicando que naquela posição há uma informação relevante (digamos, o “traço” do número 7). A representação vetorial da imagem é um vetor de 1 linha e 104 posições, sendo cada linha concatenada.

Para representação de áudios, as técnicas de processamento também são diferentes! Nossa intenção, neste momento, não é introduzir todas as técnicas de preparação de cada tipo de dado, mas de aumentar a conscientização de que essa etapa é relevante e deve ser tratada com atenção nos pipelines de Data Science baseados em modelos profundos.

A utilização de representações vetoriais para diferentes tipos de dados permite uma maior generalização dos modelos, tornando-os capazes de lidar com uma variedade de inputs. No entanto, é importante ressaltar que a qualidade e a representatividade das representações vetoriais têm um impacto significativo no desempenho dos modelos. Portanto, a escolha das técnicas de conversão de dados e a qualidade das representações vetoriais são aspectos críticos na preparação de dados para Deep Learning.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3.3 Aumentação de dados

Como mencionamos, modelos de Deep Learning são ávidos por dados, exigindo quantidades imensas de exemplos para viabilizar seu treinamento. Para aplicações de uso comum, a abundância de dados resolve essa limitação, pois atualmente, a produção e compartilhamento de dados é muito grande. No entanto, em aplicações nichadas (exemplo, aplicações médicas), a existências de MUITOS exemplos pode não ser viável.

Nesse contexto, a aumentação de dados (*Data Augmentation*) é uma técnica crucial no treinamento de modelos profundos, especialmente em tarefas de visão computacional e processamento de linguagem natural. Ela consiste em gerar novos dados de treinamento a partir dos dados existentes, aplicando transformações diversas às entradas originais. Essa abordagem aumenta a diversidade e a quantidade dos dados disponíveis para treinamento, ajudando a melhorar a generalização do modelo e a reduzir o overfitting.

Existem várias técnicas para realizar o aumento de dados em conjuntos de dados, cada uma depende do tipo de dado em questão. No **processamento de imagens**, algumas das técnicas mais comuns incluem rotação, translação, zoom, espelhamento, corte, alterações de brilho, contraste e saturação, adição de ruído e distorções geométricas. A Figura 19 ilustra algumas dessas transformações. Ainda que pareça que em vários casos a informação completa foi “destruída”, essa variabilidade nos dados é benéfica ao aprendizado.

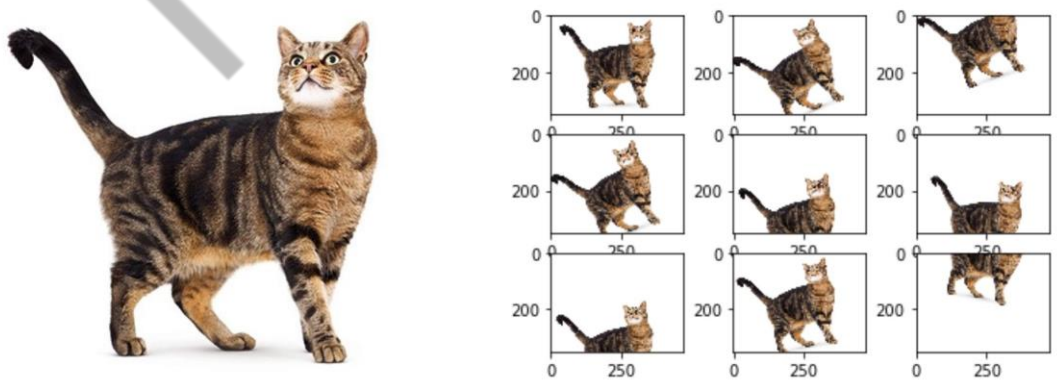

Figura 19 – Exemplo de aumentação de dados para imagens
Fonte: Imagem reproduzida de https://ubiai.tools/what-are-the-difficulties-associated-with-data-augmentation/

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

No **processamento de texto**, técnicas como substituição de palavras por sinônimos, exclusão de palavras, inversão de sequências e inserção de tokens especiais também são amplamente utilizadas (Figura 20).

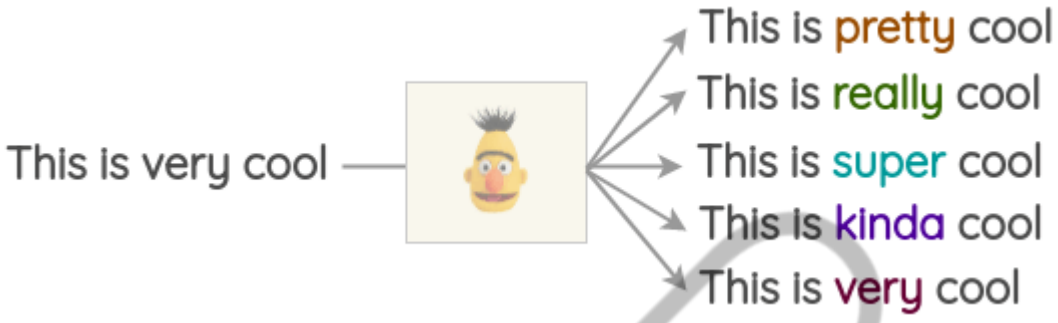

Figura 20 – Exemplo de aumentação de dados para texto
Fonte: Imagem reproduzida de https://www.analyticsvidhya.com/blog/2022/02/text-data-augmentation-in-natural-language-processing-with-texattack/

As aplicações do aumento de dados são diversas e impactantes. Essa técnica é fundamental para aproveitar ao máximo o potencial dos dados disponíveis e obter resultados mais consistentes e confiáveis em uma ampla gama de aplicações de aprendizado profundo.

4 FRAMEWORKS DE DEEP LEARNING

Assim como existem bibliotecas específicas para trabalhar com Machine Learning, como o scikit-learn e SparkML, também existem diversos frameworks para Deep Learning, essenciais para desenvolver, treinar e implantar modelos de aprendizado profundo de maneira eficiente.

Os frameworks fornecem uma série de recursos e abstrações que simplificam o processo de construção de modelos complexos e permitem a experimentação rápida com diferentes arquiteturas e técnicas. Existem diversos frameworks disponíveis, cada um com suas características e vantagens específicas. Além disso, esses frameworks estão em constante evolução, incorporando novos recursos e otimizações para atender às crescentes demandas da comunidade de aprendizado profundo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A Figura 21 mostra o estado da arte dos frameworks de DL em 2017. Desde então, a evolução foi imensa. Atualmente, entre os principais frameworks de DL, destacam-se **TensorFlow**, **PyTorch**, **Keras**, **Caffe** e **MXNet**.

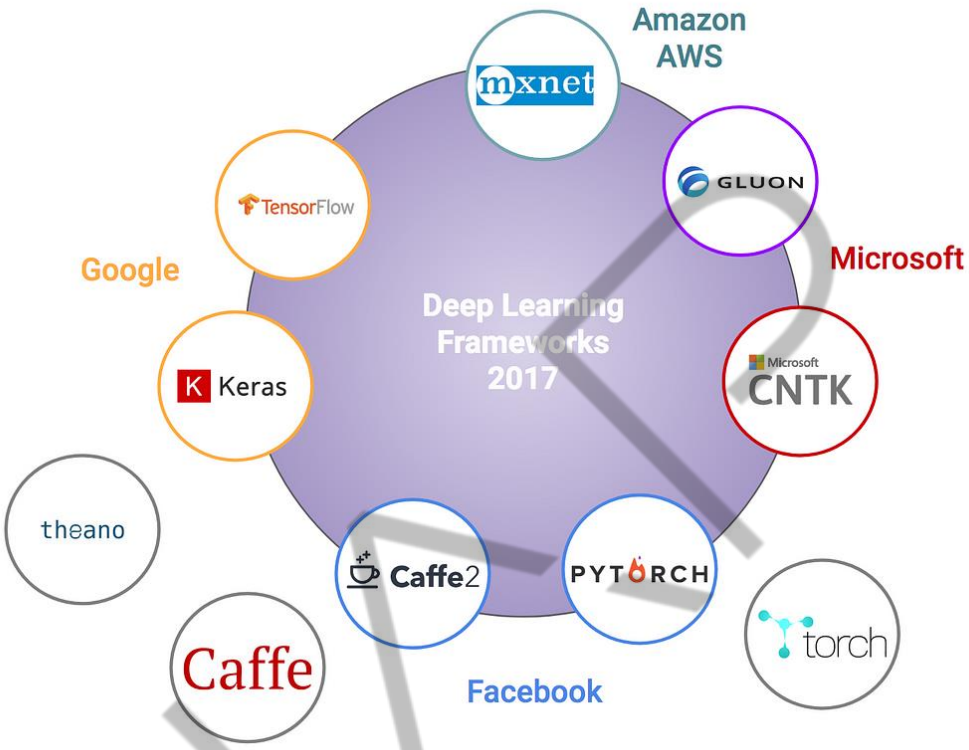

Figura 21 – Exemplo de aumentação de dados para texto
Fonte: Imagem reproduzida de https://towardsdatascience.com/battle-of-the-deep-learning-frameworks-part-i-cff0e3841750

Dentre os principais frameworks, o **TensorFlow**, desenvolvido pelo Google, é um dos mais populares e amplamente utilizados, oferecendo uma ampla gama de ferramentas para desenvolvimento de modelos, desde redes neurais convolucionais até modelos recorrentes. **PyTorch**, desenvolvido pelo Facebook, é conhecido por sua flexibilidade e simplicidade, sendo amplamente adotado pela comunidade de pesquisa. **Keras** é uma API de alto nível que pode ser executada em cima de outros frameworks, como **TensorFlow** e **Theano**, facilitando a construção rápida de protótipos e experimentos.

Desenvolvido em C++ com interface para Python, **Caffe** é conhecido por sua velocidade e eficiência, e foi frequentemente usado em aplicações de visão computacional. Sua evolução, **Caffe2**, foi introduzida em 2017 e, em 2018, foi

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

incorporada ao PyTorch. **MXNet**, mantido pela Apache, é conhecido por sua escalabilidade e eficiência em ambientes distribuídos, mas seu desenvolvimento ativo se encerrou em 2023.

A importância desses frameworks reside no fato de que eles permitem que os desenvolvedores construam e treinem modelos complexos de DL de maneira mais rápida e eficiente, sem a necessidade de implementar manualmente todos os detalhes da rede neural (imagine então de uma rede neural *profunda*!). Eles fornecem APIs intuitivas, uma variedade de módulos pré-implementados e suporte para aceleração de hardware, como GPUs, o que é essencial para lidar com conjuntos de dados grandes e complexos.

5 MÃO NA MASSA! DEEP LEARNING EM PYTHON

Agora que conhecemos o Pipeline de Deep Learning, está na hora de praticarmos sua implementação. Nessa atividade, ainda não criaremos redes neurais profundas, mas introduziremos o uso do TensorFlow, contrastando-o com o scikit-learn. Toda nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

DICA: Veja o material HOW TO para acessar o ambiente do Colab.

DICA: Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 6), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

DICA: Para essa parte do código, utilizaremos um dataset já salvo na memória do TensorFlow, por isso não há arquivos externos associados!

Utilizaremos a base “Fashion MNIST” já persistida no TensorFlow. Criada por Zalando (uma empresa de e-commerce europeia), essa base é uma coleção de imagens de artigos de moda, composta por 70.000 imagens em tons de cinza, divididas em um conjunto de treinamento de 60.000 imagens e um conjunto de teste de 10.000 imagens. Cada imagem tem uma resolução de 28x28 pixels e está associada a um label de uma das 10 classes, representando diferentes tipos de roupas e acessórios. O objetivo é criar um modelo preditivo baseado em rede neural que possa classificar corretamente as imagens em suas respectivas categorias.

Vamos começar com os *imports*:

```
import tensorflow as tf
from tensorflow import keras

from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

import numpy as np
import matplotlib.pyplot as plt
```

Código-fonte 1 – Imports necessários
Fonte: Elaborado pelo autor (2024)

A célula acima não produz saídas. Agora, vamos puxar o dataset da memória do TensorFlow. Como mencionado, já existe uma proposta de divisão das imagens em treino e teste:

```
# Carregar o dataset
fashion_mnist = keras.datasets.fashion_mnist

(train_images, train_labels), (test_images, test_labels) =
fashion_mnist.load_data()

print()
print(train_images.shape)
print(len(train_labels))
print(train_labels)
print(test_images.shape)
print(len(test_labels))
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Código-fonte 2 – Carregando o dataset Fashion MNIST
Fonte: Elaborado pelo autor (2024)

Ao executar o código acima, você verá a saída da Figura 22. A primeira linha é o formato da variável que armazena o “treino”. Ela é um array tridimensional, composto por 60 mil imagens de 28 por 28 pixels. A segunda linha é o tamanho da lista de labels do treino (e há 60 mil labels, como o esperado). A terceira linha é uma exibição parcial dos labels do treino (notem as classes sendo expressas de 0 a 9). A quarta linha é o formato da variável que armazena o “teste”. Ela é um array tridimensional, composto por 10 mil imagens de 28 por 28 pixels. Concluindo, existem 10 mil labels para as imagens de teste, conforme esperado.

```
(60000, 28, 28)
60000
[9 0 0 ... 3 0 5]
(10000, 28, 28)
10000
```

Figura 22 – Saída da execução do código-fonte 1
Fonte: Elaborado pelo autor (2024)

Para facilitar a interpretação humana dos labels, faremos uma correspondência entre cada número de 0 a 9 ao nome textual da classe. O código abaixo faz esse “DE-PARA” via posições de uma lista, contendo os nomes das classes de acordo com a documentação da base de dados. O código não produz saída.

```
# Salva as classes
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

Código-fonte 3 – Explicitando o nome dos labels do dataset
Fonte: Elaborado pelo autor (2024)

Podemos exibir algumas imagens com o código abaixo, gerando a Figura 23.

```
# Exibir algumas imagens
plt.figure(figsize=(10,10))
for i in range(25):
  plt.subplot(5, 5, i+1)
  plt.xticks([])
  plt.yticks([])
  plt.imshow(train_images[i])
  plt.xlabel(class_names[train_labels[i]])
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
plt.show()
```

Código-fonte 4 – Exibindo algumas imagens do dataset
Fonte: Elaborado pelo autor (2024)

Figura 23 – Saída da execução do código-fonte 4
Fonte: Elaborado pelo autor (2024)

Como em todo modelo de Machine Learning, é uma boa prática normalizar os dados trazendo-os para uma escala comum. Toda imagem é representada por uma matriz de pixels com valores de 0 até 255. Nesse sentido, podemos entender que as imagens já possuem dados normalizados por natureza, pois todos os pixels já estão expressos sob a mesma escala.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

No entanto, sabemos que as redes neurais convergem melhor se os dados forem escalados entre 0 e 1 ou entre -1 e +1. Já que todos os valores vão até, no máximo, 255, podemos facilmente normalizá-los dividindo-os por 255. Portanto:

```
# Normalizar os dados
train_images = train_images / 255.0
test_images = test_images / 255.0
```

Código-fonte 5 – Normalizando os dados
Fonte: Elaborado pelo autor (2024)

Criaremos 2 modelos preditivos baseados em redes neurais: o primeiro, com sklearn e o segundo, com TensorFlow. Em ambos os casos, as imagens bidimensionais precisam ser representadas como vetores unidimensionais para alimentar as redes. Esse procedimento é chamado de “achatamento” (*flattening*). Basicamente, ocorre a concatenação das linhas (ou colunas) formando um único vetor (vertical ou horizontal). Um exemplo está na Figura 24.

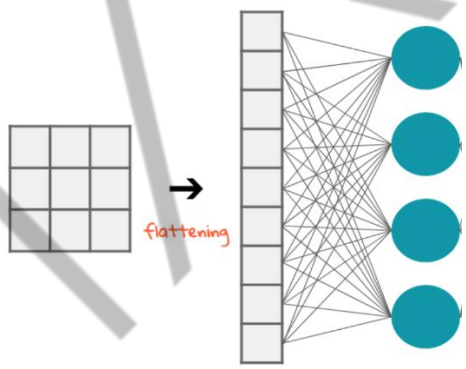

Figura 24 – Operação de achatamento de uma imagem
Fonte: Imagem reproduzida de: https://stackoverflow.com/questions/43237124/what-is-the-role-of-flatten-in-keras

Podemos fazer esse achatamento por meio do código abaixo, o qual também não gera saída:

```
# Torna as imagens lineares - Entrada para o Sklearn
def flatten(f):
  flat_list = [item for sublist in f for item in sublist]
  return flat_list

# Criar as features para uso do sklearn
train_images_features = [ flatten(image) for image in train_images ]
test_images_features = [ flatten(image) for image in test_images ]
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Código-fonte 6 – Normalizando os dados
Fonte: Elaborado pelo autor (2024)

Agora é a hora de definir o modelo e treiná-lo. Vamos começar com o método que já conhecemos: via sklearn. O código abaixo define a arquitetura da rede neural, treina o modelo e calcula a acurácia:

```
# Arquitetura da rede
ann = MLPClassifier(hidden_layer_sizes=(128,),
                    activation="relu",
                    solver="adam",
                    learning_rate_init=0.001,
                    max_iter=10,
                    random_state=42,
                    verbose=True)

# Treinamento do modelo
ann.fit(train_images_features, train_labels)

# Predição e avaliação
preds = ann.predict(test_images_features)
print("Acurácia: ", accuracy_score(test_labels, preds))
```

Código-fonte 7 – Rede neural construída com Sklearn
Fonte: Elaborado pelo autor (2024)

Para esse modelo, é esperada uma acurácia de **0,8811**. Vamos então para o modelo definido com TensorFlow:

```
# Definição da semente de aleatoriedade
tf.random.set_seed(42)

# Define o modelo
model = keras.Sequential( [
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
] )

# Compilar o modelo
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"]
              )
```

Código-fonte 8 – Rede neural construída com TensorFlow
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O código-fonte 8 indica que a forma de utilização do TensorFlow é parecida com o sklearn. A diferença é que esse framework exige que a rede neural seja explicitamente definida, camada a camada. Ela inicia com uma camada de achatamento (dispensando o tratamento feito para o sklearn), depois, temos uma camada oculta densa com 128 neurônios e uma camada de decisão densa com 10 neurônios (um para cada classe do problema). Também utilizamos a função de ativação “relu” nas camadas ocultas.

Outra diferença é que o modelo definido no TensorFlow precisa ser compilado. Nessa etapa, informamos o algoritmo de otimização de busca de pesos, a loss (função de perda) a ser empregada e qual métrica será otimizada durante o treinamento (no caso, acurácia).

Uma vantagem do TensorFlow é que é possível exibir a arquitetura da rede montada com apenas um comando:

```
# O que tem neste modelo?
model.summary()
```

Código-fonte 9 – Resumindo a rede neural construída com TensorFlow
Fonte: Elaborado pelo autor (2024)

O qual gera a saída da Figura 25, indicando que a rede possui 3 camadas e, para cada uma delas, qual o tamanho da entrada e o número de parâmetros (pesos e bias). Note que temos 107.770 parâmetros a serem aprendidos em tempo de treinamento.

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 flatten_1 (Flatten)         (None, 784)               0

 dense_2 (Dense)             (None, 128)               100480

 dense_3 (Dense)             (None, 10)                1290

=================================================================
Total params: 101770 (397.54 KB)
Trainable params: 101770 (397.54 KB)
Non-trainable params: 0 (0.00 Byte)
```

Figura 25 – Saída da execução do código-fonte 9
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Chegou a hora de treinar o modelo! De forma idêntica ao sklearn, isso é feito com o método “fit()”:

```
# Treinamento do modelo
model.fit(train_images, train_labels, epochs=10)
```

Código-fonte 10 – Treinando a rede neural construída com TensorFlow
Fonte: Elaborado pelo autor (2024)

O que gera a saída da Figura 26:

```
Epoch 1/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.4904 - accuracy: 0.8277
Epoch 2/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.3692 - accuracy: 0.8658
Epoch 3/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.3335 - accuracy: 0.8781
Epoch 4/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.3111 - accuracy: 0.8870
Epoch 5/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.2933 - accuracy: 0.8913
Epoch 6/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.2787 - accuracy: 0.8959
Epoch 7/10
1875/1875 [==============================] - 7s 4ms/step - loss: 0.2685 - accuracy: 0.9002
Epoch 8/10
1875/1875 [==============================] - 6s 3ms/step - loss: 0.2553 - accuracy: 0.9054
Epoch 9/10
1875/1875 [==============================] - 7s 3ms/step - loss: 0.2462 - accuracy: 0.9076
Epoch 10/10
1875/1875 [==============================] - 5s 3ms/step - loss: 0.2410 - accuracy: 0.9103
CPU times: user 1min 13s, sys: 5.09 s, total: 1min 18s
Wall time: 1min 3s
<keras.src.callbacks.History at 0x7a8fe421ace0>
```

Figura 26 – Saída da execução do código-fonte 10
Fonte: Elaborado pelo autor (2024)

Para encerrar, precisamos validar o modelo construído, o qual gera uma acurácia de **0,8845** (similar ao resultado passado; pequenas alterações se devem a fatores de arredondamento).

```
# Avaliação do modelo
test_loss, test_acc = model.evaluate(test_images, test_labels)

print("Test loss: ", test_loss)
print("Test  ACC: ", test_acc)
```

Código-fonte 11 – Treinando a rede neural construída com TensorFlow
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Em resumo, nessa prática guiada, aprendemos que:

- A interface de uso do TensorFlow é similar ao do Sklearn;
- No TensorFlow, precisamos definir o modelo camada a camada e compilá-lo;
- O TensorFlow permite inspecionar a arquitetura construída, apontando quantos parâmetros precisam ser ajustados durante o treinamento;
- Para redes bem simples (como esse caso), os resultados nos 2 frameworks são essencialmente iguais.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

6 HANDS ON: PRATICANDO O USO DE TENSORFLOW

6.1 Aquecimento

Neste capítulo, vimos a importância das GPUs para acelerar a computação necessária para treinamento das redes neurais profundas. Antes de começar o projeto do Capítulo, acesse o notebook abaixo e execute-o, compreendendo o que está acontecendo.

Link: https://colab.research.google.com/notebooks/gpu.ipynb

Esse notebook compara o tempo de execução de operações matriciais quando realizadas na CPU e na GPU do Google Colab, sendo que a placa gráfica (GPU) pode ser até 35x mais rápida!

O notebook também orienta como configurar o Google Colab para ativar ambientes com GPU. Isso será particularmente útil para treinar redes profundas.

DICA: Para alterar seu ambiente pra contendo uma GPU no Google Colab, basta clicar no ambiente de execução, depois em "Alterar o tipo de ambiente de execução" e então selecionar "T4 GPU" como acelerador de hardware.

Outra informação importante é como indicar ao código em qual dispositivo ele será executado, sendo:

- "`with tf.device('/cpu:0')`": força a execução na CPU;
- "`with tf.device('/device:GPU:0')`": força a execução na GPU.

6.2 Seu desafio

Sua tarefa é criar um modelo de classificação de números escritos a mão, baseado nos dados do MNIST, o qual também está disponível no TensorFlow. A base MNIST (*Modified National Institute of Standards and Technology database*) é um

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

conjunto de dados utilizado para treinamento e teste em campo de visão computacional. Consiste em 70.000 imagens em escala de cinza de dígitos manuscritos (0 a 9), divididas em um conjunto de treino de 60.000 imagens e um conjunto de teste de 10.000 imagens. Cada imagem tem uma resolução de 28x28 pixels. O objetivo da tarefa de machine learning com a base MNIST é corretamente classificar as imagens dos dígitos de 0 a 9.

Você deve:

- Pesquisar como ler a base MNIST disponível no TensorFlow;
- Definir a arquitetura usando o TensorFlow;
- Treinar o modelo usando GPU.

Espera-se uma Acurácia de pelo menos 0,95.

6.3 Proposta de solução

Abaixo, temos uma proposta de solução para o desafio apresentado. Essa é apenas uma forma de resolver o problema. Seus modelos podem ser diferentes e com melhores/piores resultados: o importante é praticar o uso de TensorFlow e a vantagem de treinar os modelos na GPU.

```
# Import de bibliotecas necessárias
import numpy as np
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

# Carregando a base de dados MNIST
mnist = tf.keras.datasets.mnist

# Carregando os conjuntos de treino e teste
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Exploração inicial dos dados
print("Shape dos dados de treino:", x_train.shape)
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
print("Shape dos dados de teste:", x_test.shape)

# Normalização dos dados para valores de pixel entre 0 e 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# Visualização de um grid de 10 linhas (uma para cada dígito) e
# 15 colunas (com imagens selecionadas aleatoriamente)
num_linhas = 10
num_colunas = 15
plt.figure(figsize=(15, 10))

for i in range(num_linhas):
    indices = np.where(y_train == i)[0]
    for j in range(num_colunas):
        # Escolher aleatoriamente um índice dos exemplos com o rótulo
atual
        idx = np.random.choice(indices)

        plt.subplot(num_linhas, num_colunas, i * num_colunas + j + 1)
        plt.imshow(x_train[idx], cmap='gray')
        plt.axis('off')

plt.tight_layout()
plt.show()

# Construção do modelo de rede neural
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

# Compilando o modelo
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Treinando o modelo na GPU
with tf.device('/device:GPU:0'):
  history = model.fit(x_train, y_train, epochs=10,
validation_split=0.2)

# Avaliando o modelo no conjunto de teste
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print('\nACC Teste:', test_acc)

# Plot da acurácia de treinamento e validação
plt.plot(history.history['accuracy'], label='Acurácia (treino )')
```

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
plt.plot(history.history['val_accuracy'], label='Acurácia
(validação)')
plt.title('ACC do Treino e da Validação')
plt.ylabel('Acurácia')
plt.xlabel('No. épocas')
plt.legend(loc="upper left")
plt.show()
```

Código-fonte 12 – Possível solução do desafio Hands On
Fonte: Elaborado pelo autor (2024)

A saída do código-fonte 12 gera a seguinte informação textual:

- `Shape dos dados de treino: (60000, 28, 28)`
- `Shape dos dados de teste: (10000, 28, 28)`

```
Epoch 1/10
1500/1500 [==============================] - 7s 4ms/step - loss: 0.2901
- accuracy: 0.9167 - val_loss: 0.1586 - val_accuracy: 0.9554
Epoch 2/10
1500/1500 [==============================] - 5s 3ms/step - loss: 0.1284
- accuracy: 0.9627 - val_loss: 0.1133 - val_accuracy: 0.9677
Epoch 3/10
1500/1500 [==============================] - 6s 4ms/step - loss: 0.0879
- accuracy: 0.9734 - val_loss: 0.1033 - val_accuracy: 0.9703
Epoch 4/10
1500/1500 [==============================] - 4s 3ms/step - loss: 0.0645
- accuracy: 0.9802 - val_loss: 0.0880 - val_accuracy: 0.9747
Epoch 5/10
1500/1500 [==============================] - 5s 3ms/step - loss: 0.0484
- accuracy: 0.9854 - val_loss: 0.0894 - val_accuracy: 0.9752
Epoch 6/10
1500/1500 [==============================] - 5s 4ms/step - loss: 0.0377
- accuracy: 0.9889 - val_loss: 0.0880 - val_accuracy: 0.9747
Epoch 7/10
1500/1500 [==============================] - 5s 3ms/step - loss: 0.0293
- accuracy: 0.9915 - val_loss: 0.0889 - val_accuracy: 0.9756
Epoch 8/10
1500/1500 [==============================] - 5s 4ms/step - loss: 0.0236
- accuracy: 0.9933 - val_loss: 0.0923 - val_accuracy: 0.9747
Epoch 9/10
1500/1500 [==============================] - 5s 3ms/step - loss: 0.0184
- accuracy: 0.9948 - val_loss: 0.0955 - val_accuracy: 0.9747
Epoch 10/10
1500/1500 [==============================] - 5s 3ms/step - loss: 0.0163
- accuracy: 0.9951 - val_loss: 0.0937 - val_accuracy: 0.9766
313/313 - 1s - loss: 0.0835 - accuracy: 0.9764 - 628ms/epoch - 2ms/step
```

- `ACC Teste: 0.9764000177383423`

E a visualização dos exemplos de dados (Figura 27) e das curvas de aprendizado (Figura 28).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

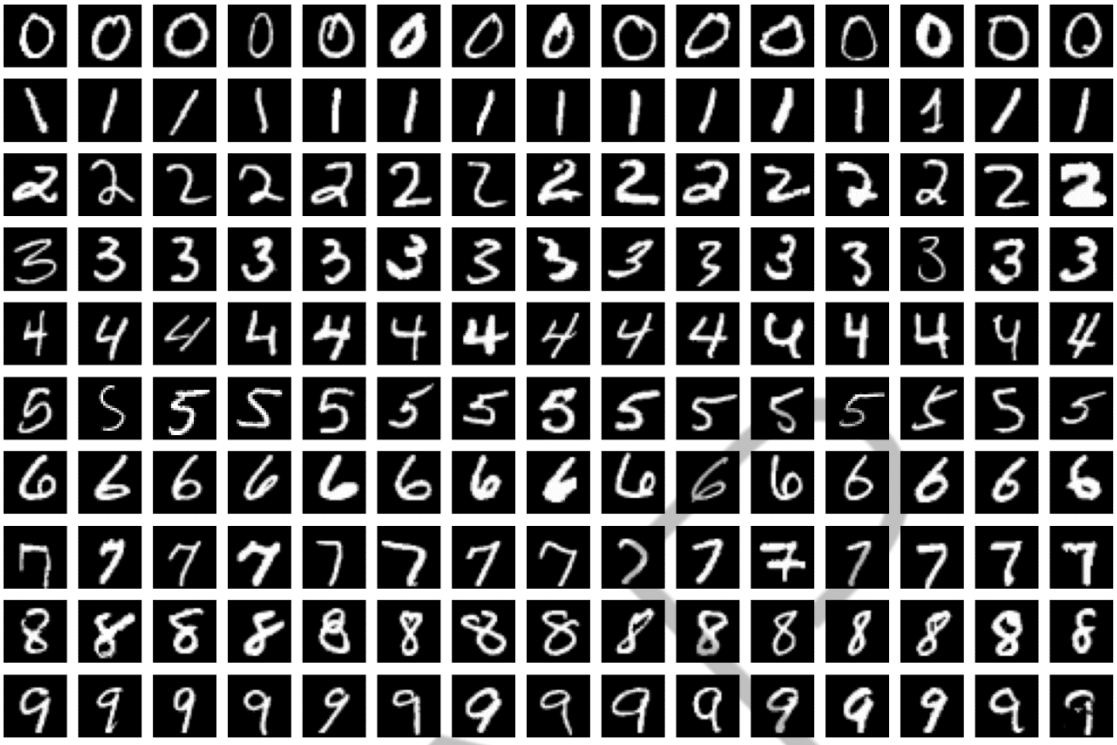

Figura 27 – Saída da execução do código-fonte 12 – Parte A
Fonte: Elaborado pelo autor (2024)

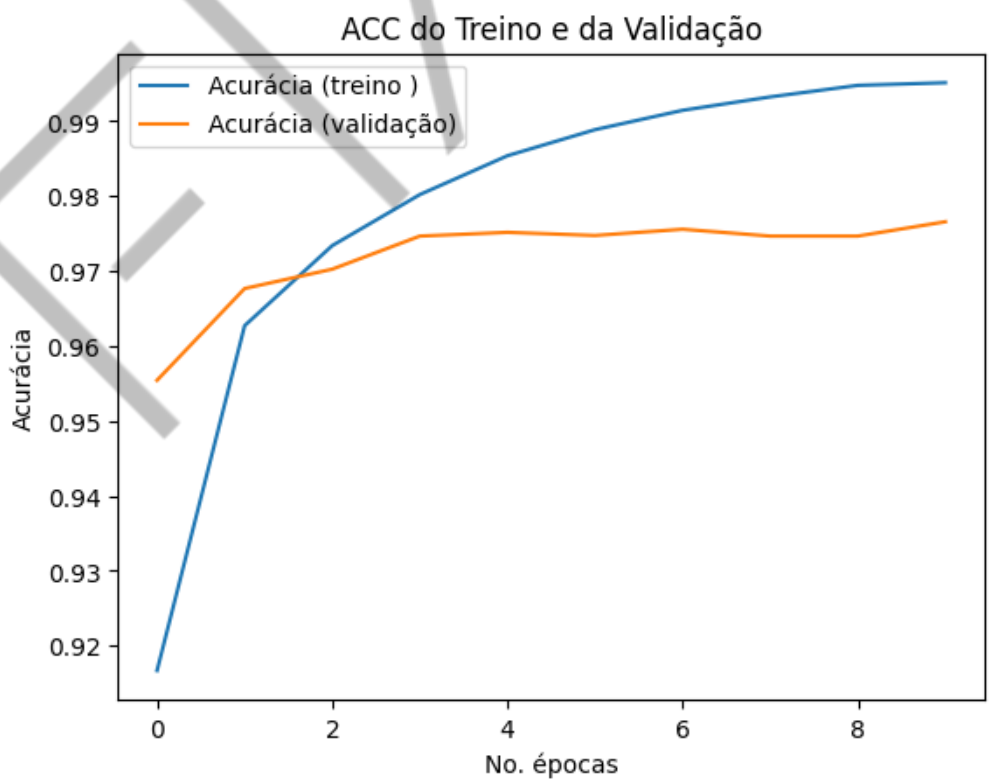

Figura 28 – Saída da execução do código-fonte 12 – Parte B
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

CALIN, Ovidiu. **Deep learning architectures**. New York City: Springer International Publishing, 2020.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep learning**. MIT press, 2016.

HAYKIN, Simon. **Redes neurais:** princípios e prática. Bookman Editora, 2008.

MISHRA, Ranjan Kumar; REDDY, GY Sandesh; PATHAK, Himanshu. **The understanding of deep learning:** A comprehensive review. Mathematical Problems in Engineering, 2021, 2021: 1-15.

WANI, M. Arif; KHOSHGOFTAAR, Taghi M.; PALADE, Vasile (ed.). **Deep Learning Applications**, Volume 3. Springer, 2021.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

GLOSSÁRIO

Termo	Explicação
CNN	“Convolutional neural network” (Redes Neurais Convolucionais)
CPU	“Central Processing Unit” (Unidade de Processamento Central)
DL	Deep Learning
FLOPS	“Floating-point Operations Per Second” (Operações de Ponto Flutuante por Segundo)
GANs	“Generative Adversarial Network” (Redes Adversárias Generativas)
GenAI	“Generative AI” (Inteligência Artificial Generativa)
GPU	“Graphics Processing Unit” (Unidade de Processamento Gráfico)
GRUs	“Gated Recurrent Units” (Unidades Recorrentes Fechadas)
IA	Inteligência Artificial
ILSVRC	ImageNet Large Scale Visual Recognition Challenge
IoT	Internet of Things
LSTMs	“Long Short-Term Memory” (Memória de Curto Longo Prazo)
ML	Machine Learning
PLN	Processamento de Linguagem Natural

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com